

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Data Cleaning Challenge

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0503

COURSE OUTCOME COVERED

CO3: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex.

Assessment Tool Weightage: 35%

Total Marks: 20

Code of Conduct

I Naresh M(192424381) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Naresh M

Debugging

Criteria	Anomaly Detection (4 Marks)	Cleaning Methods (4 Marks)	Data Transformation (4 Marks)	Analytical Comparison (4 Marks)	Governance Discussion (4 Marks)
Marks					

Total Marks Awarded:

Questions:

Question 1: E-Commerce Customer Transactions Dataset

An online retail company has collected customer transaction data from multiple sales channels. The management wants to identify the top-performing customer segments and products. However, the dataset contains several quality issues.

CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City
C101	Raj Kumar	Male	28	Electronics	2	15000 INR	10-01-2025	Chennai
C102	Priya S	Female	35	Fashion	3	1200 INR	11-01-2025	chennai
C103	Arun V	Male	150	Electronics	1	300 USD	12-01-2025	CHENNAI
C104	Kavya R	Female	0	Grocery	-2	800 INR	15-01-2025	Coimbatore
C105	Rahul P	Male	42	Fashion	5	2500 INR	18-01-2025	Madurai
C105	Rahul P	Male	42	Fashion	5	2500 INR	18-01-2025	Madurai
NULL	Sneha M	Female	30	Electronics	2	450 USD	05-02-2025	Trichy
C107	Deepak N	Male		Grocery	3	600 INR	07-02-2025	Tiruchirappalli
C108	Meena K	Female	27	Fashion	1	1800 INR	2025-13-01	Salem
C109	John D	Male	31	Electronics	4	250 EUR	15-02-2025	Chennai

Data Issues

- Missing Customer IDs
- Duplicate transactions
- Negative quantities
- Unit prices recorded in different currencies
- Age values such as 0, 150, and blanks
- Inconsistent city names (e.g., Chennai, chennai, CHENNAI)
- Incorrect purchase dates

Tasks

1. Identify and classify all data quality issues. (4 Marks)

2. Design a cleaning strategy for each issue. (4 Marks)
3. Perform data standardization and transformation. (4 Marks)
4. Analyze how cleaning affects total revenue calculations. (4 Marks)
5. Generate a report discussing business risks if the data were not cleaned (4 Marks)

Question 2: Hospital Patient Records Dataset

A hospital wants to analyze patient admissions and treatment effectiveness. Data were entered manually by different departments, leading to inconsistencies.

PatientID	Name	Age	Gender	AdmissionDate	DischargeDate	Diagnosis	TreatmentCost	DoctorID
P1001	Ravi Kumar	45	M	10-01-2025	15-01-2025	Diabetes	25000	D101
P1002	Priya S	38	F	12-01-2025	11-01-2025	Hypertension	18000	D102
P1003	Arun V	125	M	13-01-2025	20-01-2025	diabetes	22000	D101
NULL	Kavya R	29	F	15-01-2025	17-01-2025	Diabtes	15000	D103
P1005	Rahul P	51	M	18-01-2025	25-01-2025	Asthma	NULL	D104
P1005	Rahul P	51	M	18-01-2025	25-01-2025	Asthma	NULL	D104
P1006	Sneha M	34	F	15-01-2025	18-01-2025	Hypertension	12000	D102
P1007	Deepak N		M	01-02-2025	05-02-2025	Asthma	10000	D104
P1008	Meena K	26	F	10-02-2025	15-02-2025	HYPERTENSION	13500	D102
P1009	John D	40	M	12-02-2025	18-02-2025	Diabetes	20000	D101

Data Issues

- Missing patient IDs

- Duplicate patient records
- Discharge dates before admission dates
- Null treatment costs
- Inconsistent diagnosis names
- Multiple formats of dates
- Unrealistic patient ages (>120)

Tasks

1. Detect all inconsistencies and anomalies. (4 Marks)
2. Develop rules for handling missing and invalid values. (4 Marks)
3. Create a process to standardize diagnosis names. (4 Marks)
4. Evaluate how cleaning changes average treatment cost and hospital stay duration. (4 Marks)
5. Recommend data governance measures to prevent future errors. (4 Marks)

Question 3: Banking Loan Approval Dataset

A bank uses customer data to determine loan eligibility. The dataset contains errors that may affect risk assessment and approval decisions.

CustomerID	AnnualIncome	CreditScore	LoanAmount	EmploymentStatus	Age	ExistingDebt	LoanStatus
L001	750000	720	300000	Full Time	35	50000	Approved
L002	900K	680	400000	FT	42	100000	Approved
L003	1200000	NULL	600000	fulltime	28	200000	Approved
L004	150000	450	-200000	Part Time	30	50000	Rejected
L005	5000000	790	1000000	Self Employed	16	10000	Approved
L006	450000	350	250000	Unemployed	40	300000	Rejected
L007	75000000	800	500000	Full Time	45	100000	Approved
L008	650000	690	350000	FT	33	70000	Approved

L008	650000	690	350000	FT	33	70000	Approved
L010	NULL	710	300000	Full Time	37	80000	Approved

Data Issues

- Missing credit scores
- Income values recorded in different units
- Negative loan amounts
- Duplicate customer IDs
- Age values below 18
- Employment status entered differently (FT, Full Time, fulltime)
- Extreme outliers in income

Tasks

1. Identify critical issues that could impact loan decisions. (4 Marks)
2. Propose cleaning techniques for missing and inconsistent data. (4 Marks)
3. Detect and justify treatment of outliers. (4 Marks)
4. Analyze the effect of cleaning on loan approval rates. (4 Marks)
5. Discuss ethical implications of incorrect data cleaning decisions. (4 Marks)

Question 4: Smart City Traffic Monitoring Dataset

A smart city project collects traffic sensor data to optimize traffic signals. Sensor malfunctions and transmission errors have affected data quality.

SensorID	Timestamp	VehicleCount	AverageSpeed	TrafficSignalStatus	RoadName	WeatherCondition
S101	01-01-2025 08:00	120	45	Green	Anna Salai	Sunny
S102	01-01-2025 08:05	135	52	Red	anna salai	Sunny
S103	01-01-2025 08:10	-50	48	Green	ANNA SALAI	Cloudy
S104	01-01-2025 08:15	140	350	Yellow	GST Road	Rainy
S105	NULL	110	42	Green	GST ROAD	Sunny
S106	01-01-2025 08:25	130	46	Red	OMR	NULL
S106	01-01-2025 08:25	130	46	Red	OMR	NULL
S107	01-01-2025 08:30	150	55	Green	Old Mahabalipuram Road	Sunny
S10A	01-01-2025 08:35	145	53	Green	OMR	Cloudy
S109	01-01-2025 08:40	138	50	Green	GST Road	Rainy

Data Issues

- Missing timestamps
- Duplicate sensor readings
- Vehicle counts recorded as negative numbers
- Average speed exceeding 300 km/h
- Missing weather information
- Inconsistent road naming conventions
- Sensor IDs with formatting errors

Tasks

1. Assess the quality of the traffic dataset. (4 Marks)
2. Recommend cleaning techniques for time-series data. (4 Marks)
3. Design rules to identify faulty sensor readings. (4 Marks)
4. Evaluate how cleaning affects congestion analysis. (4 Marks)
5. Suggest automated monitoring techniques for future data collection. (4 Marks)

Question 5: Global Sales Performance Dataset

A multinational company has merged sales data from five countries. Management wants to evaluate regional performance and forecast future sales. The merged dataset contains severe data quality problems.

Region	Country	SalesPerson	Product	SalesAmount	Currency	SalesDate	CustomerRating
Asia	India	Raj Kumar	Laptop	50000	INR	10-01-2025	4
Asia	India	Rajkumar	Laptop	55000	INR	10-01-2025	5
Europe	Germany	Anna M	Laptop	1200	EUR	12-01-2025	4
Europe	Germany	Anna M	Laptop	1200	EUR	12-01-2025	4
North America	USA	John D	Laptop	1500	USD	01-15-2025	6
Asia	India	Priya S	Mobile	NULL	INR	18-01-2025	3
Europe	France	Pierre L	Ordinateur Portable	1300	EUR	20-01-2025	4
Asia	India	Rahul P	Mobile	9999999	INR	25-01-2025	5
UK	UK	David T	Laptop	1100	GBP	28-01-2025	2
Asia	India	Priya S	Smartphone	35000	INR	30-01-2025	0

Data Issues

- Multiple currencies (USD, EUR, INR, GBP)
- Missing sales amounts
- Duplicate regional entries
- Customer ratings outside the scale (1–5)
- Different date formats across countries
- Product names in multiple languages
- Salespersons recorded with spelling variations
- Extreme sales outliers due to data-entry errors

Tasks

1. Develop a comprehensive data-cleaning framework for multinational datasets. (4 Marks)
2. Design methods to standardize currencies, products, and dates. (4 Marks)
3. Handle missing values and outliers using appropriate analytical techniques. (4 Marks)
4. Compare regional sales rankings before and after cleaning. (4 Marks)
5. Prepare an executive report explaining how data quality influences strategic business decisions. (4 Marks)

Signature of the Course Faculty

Question 1: E-Commerce Customer Transactions Dataset

Tasks

- 1) Detect all inconsistencies and anomalies. (4 Marks)

Code :

```
import pandas as pd
df = pd.read_csv("ecom.csv")
print("1. DATA QUALITY ISSUES")
print("-" * 60)
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Records:")
print(df[df.duplicated(keep=False)])
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
print("\nNegative Quantities:")
print(df[df["Quantity"] < 0])
print("\nInvalid Ages:")
print(df[(df["Age"] <= 0) | (df["Age"] > 120)])
print("\nCurrency Values:")
currency = df["UnitPrice"].astype(str).str.extract(
    r"([A-Za-z]+)$"
)[0]
print(currency.value_counts())
print("\nCity Names:")
print(df["City"].unique())
dates = pd.to_datetime(
    df["PurchaseDate"],
    errors="coerce",
    dayfirst=True
```



```
)
print("\nInvalid Purchase Dates:")
print(df[dates.isna()])
```

Output:

```
===== RESTART: C:/Users/nares/Videos/dsa0503/experiments/as3.1.1.py =====
1. DATA QUALITY ISSUES
-----

Missing Values:
CustomerID      1
CustomerName    0
Gender          0
Age             1
ProductCategory 0
Quantity        0
UnitPrice       0
PurchaseDate    0
City            0
dtype: int64

Duplicate Records:
  CustomerID CustomerName Gender ... UnitPrice PurchaseDate City
4         C105      Rahul P   Male ...    2500      18-01-2025 Madurai
5         C105      Rahul P   Male ...    2500      18-01-2025 Madurai

[2 rows x 9 columns]

Negative Quantities:
  CustomerID CustomerName Gender ... UnitPrice PurchaseDate City
3         C104      Kavya R   Female ...     800      15-01-2025 Coimbatore

[1 rows x 9 columns]

Invalid Ages:
  CustomerID CustomerName Gender ... UnitPrice PurchaseDate City
2         C103      Arun V   Male ...     300      12-01-2025 CHENNAI
3         C104      Kavya R   Female ...     800      15-01-2025 Coimbatore

[2 rows x 9 columns]

Currency Values:
0
INR    7
USD    2
EUR    1
Name: count, dtype: int64
```

```

City Names:
<StringArray>
[      'Chennai',      'chennai',      'CHENNAI',      'Coimbatore',
      'Madurai',      'Trichy', 'Tiruchirappalli',      'Salem']
Length: 8, dtype: str

Invalid Purchase Dates:
   CustomerID CustomerName Gender  Age  ... Quantity  UnitPrice PurchaseDate  City
8          C108      Meena K  Female  27.0  ...         1      1800 INR   2025-13-01  Salem

[1 rows x 9 columns]

```

2) Design a cleaning strategy for each issue. (4 Marks)

Code:

```

import pandas as pd
df = pd.read_csv("ecom.csv")
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
print("2. CLEANING STRATEGY")
print("-" * 70)
strategy = pd.DataFrame({
    "Data Quality Issue": [
        "Missing Customer ID",
        "Duplicate Transactions",
        "Negative Quantity",
        "Different Currencies",
        "Invalid Age (0 or >120)",
        "Missing Age",
        "Inconsistent City Names",
        "Incorrect Purchase Date"
    ],
    "Cleaning Method": [
        "Verify customer details and assign a valid Customer ID",
        "Identify and remove duplicate transaction records",
        "Verify transaction and correct the quantity or mark it as invalid",
        "Standardize all prices to a common currency",
        "Replace invalid age with missing value and verify from source",
        "Fill using verified customer information; otherwise keep as missing",
        "Convert all city names to a standard format",
        "Convert to valid date; invalid dates are marked as missing"
    ]
})

print(strategy.to_string(index=False))

```

Output:

2. CLEANING STRATEGY

Data Quality Issue	Cleaning Method
Missing Customer ID	Verify customer details and assign a valid Customer ID
Duplicate Transactions	Identify and remove duplicate transaction records
Negative Quantity	Verify transaction and correct the quantity or mark it as invalid
Different Currencies	Standardize all prices to a common currency
Invalid Age (0 or >120)	Replace invalid age with missing value and verify from source
Missing Age	Fill using verified customer information; otherwise keep as missing
Inconsistent City Names	Convert all city names to a standard format
Incorrect Purchase Date	Convert to valid date; invalid dates are marked as missing

3. Perform data standardization and transformation. (4 Marks)

Code:

```
import pandas as pd
df = pd.read_csv("ecom.csv")
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
df["City"] = df["City"].str.strip().str.title()
df.loc[
    (df["Age"] <= 0) | (df["Age"] > 120),
    "Age"
] = pd.NA
df.loc[
    df["Quantity"] < 0,
    "Quantity"
] = pd.NA
df["Currency"] = df["UnitPrice"].astype(str).str.extract(
    r"([A-Za-z]+)$"
)[0]
df["UnitPrice"] = (
    df["UnitPrice"]
    .astype(str)
    .str.extract(r"([0-9]+\.[0-9]*)")[0]
)
df["UnitPrice"] = pd.to_numeric(
    df["UnitPrice"],
    errors="coerce"
)
df["PurchaseDate"] = pd.to_datetime(
    df["PurchaseDate"],
    errors="coerce",
    dayfirst=True
)
df["Revenue"] = df["Quantity"] * df["UnitPrice"]
print("3. STANDARDIZED AND TRANSFORMED DATA")
print("-" * 80)
print(df.to_string(index=False))
```

Output:

```
===== RESTART: C:/Users/nares/videos/dsa0503/experiments/ass.1.1.3.py
3. STANDARDIZED AND TRANSFORMED DATA
-----
CustomerID CustomerName Gender Age ProductCategory Quantity UnitPrice PurchaseDate City Currency Revenue
C101 Raj Kumar Male 28.0 Electronics 2.0 15000 2025-01-10 Chennai INR 30000.0
C102 Priya S Female 35.0 Fashion 3.0 1200 2025-01-11 Chennai INR 3600.0
C103 Arun V Male NaN Electronics 1.0 300 2025-01-12 Chennai USD 300.0
C104 Kavya R Female NaN Grocery NaN 800 2025-01-15 Coimbatore INR NaN
C105 Rahul P Male 42.0 Fashion 5.0 2500 2025-01-18 Madurai INR 12500.0
C105 Rahul P Male 42.0 Fashion 5.0 2500 2025-01-18 Madurai INR 12500.0
NaN Sneha M Female 30.0 Electronics 2.0 450 2025-02-05 Trichy USD 900.0
C107 Deepak N Male NaN Grocery 3.0 600 2025-02-07 Tiruchirappalli INR 1800.0
C108 Meena K Female 27.0 Fashion 1.0 1800 NaT Salem INR 1800.0
C109 John D Male 31.0 Electronics 4.0 250 2025-02-15 Chennai EUR 1000.0
```

4. Analyze how cleaning affects total revenue calculations. (4 Marks)

Code:

```
import pandas as pd

# Load the CSV file
df = pd.read_csv("ecom.csv")

# -----
# Convert Quantity to numeric
# -----

df["Quantity"] = pd.to_numeric(
    df["Quantity"], errors="coerce"
)

# -----
# Extract Currency from UnitPrice
# -----

df["Currency"] = df["UnitPrice"].astype(str).str.extract(
    r"([A-Za-z]+)$"
)[0]
```

```

# -----

# Extract numeric UnitPrice

# -----

df["UnitPrice"] = (
    df["UnitPrice"]
    .astype(str)
    .str.extract(r"([0-9]+\.[0-9]*)")[0]
)

df["UnitPrice"] = pd.to_numeric(
    df["UnitPrice"], errors="coerce"
)

# -----

# Calculate Revenue BEFORE cleaning

# -----

df["Revenue_Before"] = (
    df["Quantity"] * df["UnitPrice"]
)

print("4. REVENUE ANALYSIS")

print("-" * 60)

print("\nRevenue Before Cleaning:")

print(
    df.groupby("Currency")["Revenue_Before"].sum()

```

)

```
# =====
```

```
# CLEANING
```

```
# =====
```

```
cleaned_df = df.copy()
```

```
# Remove duplicate transactions
```

```
cleaned_df = cleaned_df.drop_duplicates(
```

```
    subset=[
```

```
        "CustomerID",
```

```
        "CustomerName",
```

```
        "Gender",
```

```
        "Age",
```

```
        "ProductCategory",
```

```
        "Quantity",
```

```
        "UnitPrice",
```

```
        "Currency",
```

```
        "PurchaseDate",
```

```
        "City"
```

```
    ]
```

```
)
```

```
# Remove invalid negative quantity
```

```
cleaned_df.loc[
```

```
cleaned_df["Quantity"] < 0,  
"Quantity"  
] = pd.NA
```

```
# Calculate revenue AFTER cleaning
```

```
cleaned_df["Revenue_After"] = (  
    cleaned_df["Quantity"] *  
    cleaned_df["UnitPrice"]  
)
```

```
print("\nRevenue After Cleaning:")
```

```
print(  
    cleaned_df.groupby("Currency")["Revenue_After"].sum()  
)
```

```
# -----
```

```
# Compare revenue
```

```
# -----
```

```
before = df.groupby("Currency")["Revenue_Before"].sum()
```

```
after = cleaned_df.groupby("Currency")["Revenue_After"].sum()
```

```
comparison = pd.DataFrame({  
    "Before Cleaning": before,  
    "After Cleaning": after
```

```
})
```

```
comparison["Difference"] = (  
    comparison["Before Cleaning"] -  
    comparison["After Cleaning"]  
)
```

```
print("\nRevenue Comparison:")  
print(comparison)
```

Output:

4. REVENUE ANALYSIS

Revenue Before Cleaning:

Currency

EUR 1000

INR 60600

USD 1200

Name: Revenue_Before, dtype: int64

Revenue After Cleaning:

Currency

EUR 1000.0

INR 49700.0

USD 1200.0

Name: Revenue_After, dtype: float64

Revenue Comparison:

	Before Cleaning	After Cleaning	Difference
Currency			
EUR	1000	1000.0	0.0
INR	60600	49700.0	10900.0
USD	1200	1200.0	0.0

5. Generate a report discussing business risks if the data were not cleaned. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("ecom.csv")
```



```

print("5. BUSINESS RISK REPORT")

print("-" * 80)

risk_report = pd.DataFrame({

    "Data Quality Issue": [

        "Missing Customer ID",

        "Duplicate Transactions",

        "Negative Quantity",

        "Different Currencies",

        "Invalid Age Values",

        "Missing Age",

        "Inconsistent City Names",

        "Incorrect Purchase Date"

    ],

    "Business Risk": [

        "Customer transactions cannot be correctly identified or tracked.",

        "Sales and revenue may be counted twice, resulting in overstated performance.",

        "Product quantity and revenue calculations may become incorrect.",

        "Revenue comparison between currencies may produce misleading results.",

        "Customer segmentation and demographic analysis may become inaccurate.",

        "Customer analysis based on age may be incomplete.",

        "City-wise sales analysis may treat the same city as different locations.",

        "Monthly, seasonal and date-based sales analysis may become inaccurate."

    ]

})

print(risk_report.to_string(index=False))

print("\n" + "=" * 80)

```

```
print("OVERALL BUSINESS IMPACT")
```

```
print("=" * 80)
```

```
print("""
```

1. Revenue Risk:

Duplicate and negative transactions can cause incorrect revenue calculations.

2. Customer Analysis Risk:

Missing and invalid customer information can lead to incorrect customer segmentation and targeting.

3. Product Analysis Risk:

Incorrect quantities can affect product sales and performance analysis.

4. Regional Analysis Risk:

Inconsistent city names and different currencies can produce misleading regional sales comparisons.

5. Decision-Making Risk:

Management may make incorrect business decisions because of inaccurate reports and unreliable data.

```
""")
```

Output:

5. BUSINESS RISK REPORT

Data Quality Issue	Business Risk
Missing Customer ID	Customer transactions cannot be correctly identified or tracked.
Duplicate Transactions	Sales and revenue may be counted twice, resulting in overstated performance.
Negative Quantity	Product quantity and revenue calculations may become incorrect.
Different Currencies	Revenue comparison between currencies may produce misleading results.
Invalid Age Values	Customer segmentation and demographic analysis may become inaccurate.
Missing Age	Customer analysis based on age may be incomplete.
Inconsistent City Names	City-wise sales analysis may treat the same city as different locations.
Incorrect Purchase Date	Monthly, seasonal and date-based sales analysis may become inaccurate.

OVERALL BUSINESS IMPACT

1. Revenue Risk:
Duplicate and negative transactions can cause incorrect revenue calculations.
2. Customer Analysis Risk:
Missing and invalid customer information can lead to incorrect customer segmentation and targeting.
3. Product Analysis Risk:
Incorrect quantities can affect product sales and performance analysis.
4. Regional Analysis Risk:
Inconsistent city names and different currencies can produce misleading regional sales comparisons.
5. Decision-Making Risk:
Management may make incorrect business decisions because of inaccurate reports and unreliable data.

Question 2: Hospital Patient Records Dataset

1. Detect all inconsistencies and anomalies. (4 Marks)

Code:

```
import pandas as pd

df = pd.read_csv("hospital.csv")

print("1. INCONSISTENCIES AND ANOMALIES")

print("-" * 70)

df["Age"] = pd.to_numeric(df["Age"], errors="coerce")

df["TreatmentCost"] = pd.to_numeric(
    df["TreatmentCost"], errors="coerce"
)

print("\nMissing Patient IDs:")

print(df[df["PatientID"].isna()])
```

```

print("\nDuplicate Patient Records:")
print(df[df.duplicated(keep=False)])

df["AdmissionDate"] = pd.to_datetime(
    df["AdmissionDate"],
    errors="coerce",
    dayfirst=True
)

df["DischargeDate"] = pd.to_datetime(
    df["DischargeDate"],
    errors="coerce",
    dayfirst=True
)

invalid_dates = df[
    df["DischargeDate"] < df["AdmissionDate"]
]

print("\nDischarge Date Before Admission Date:")
print(invalid_dates)

print("\nNull Treatment Costs:")
print(df[df["TreatmentCost"].isna()])

print("\nDiagnosis Values:")
print(df["Diagnosis"].unique())

print("\nUnrealistic Patient Ages:")
print(df[df["Age"] > 120])

print("\nSUMMARY OF ISSUES")
print("-" * 70)

print("Missing Patient IDs      :", df["PatientID"].isna().sum())
print("Duplicate Records        :", df.duplicated().sum())
print("Invalid Date Records      :", len(invalid_dates))

```

```
print("Null Treatment Costs      :", df["TreatmentCost"].isna().sum())
```

```
print("Unrealistic Ages         :", (df["Age"] > 120).sum())
```

Output:

1. INCONSISTENCIES AND ANOMALIES

Missing Patient IDs:

PatientID	Name	Age	...	Diagnosis	TreatmentCost	DoctorID	
3	NaN	Kavya R	29.0	...	Diabetes	15000.0	D103

[1 rows x 9 columns]

Duplicate Patient Records:

PatientID	Name	Age	...	Diagnosis	TreatmentCost	DoctorID	
4	P1005	Rahul P	51.0	...	Asthma	NaN	D104
5	P1005	Rahul P	51.0	...	Asthma	NaN	D104

[2 rows x 9 columns]

Discharge Date Before Admission Date:

PatientID	Name	Age	...	Diagnosis	TreatmentCost	DoctorID	
1	P1002	Priya S	38.0	...	Hypertension	18000.0	D102

[1 rows x 9 columns]

Null Treatment Costs:

PatientID	Name	Age	...	Diagnosis	TreatmentCost	DoctorID	
4	P1005	Rahul P	51.0	...	Asthma	NaN	D104
5	P1005	Rahul P	51.0	...	Asthma	NaN	D104

[2 rows x 9 columns]

Diagnosis Values:

<StringArray>

['Diabetes', 'Hypertension', 'diabetes', 'Diabetes', 'Asthma', 'HYPERTENSION']

Length: 6, dtype: str

Unrealistic Patient Ages:

PatientID	Name	Age	...	Diagnosis	TreatmentCost	DoctorID	
2	P1003	Arun V	125.0	...	diabetes	22000.0	D101

[1 rows x 9 columns]

SUMMARY OF ISSUES

Missing Patient IDs	: 1
Duplicate Records	: 1
Invalid Date Records	: 1
Null Treatment Costs	: 2
Unrealistic Ages	: 1

2. Develop rules for handling missing and invalid values. (4 Marks)

Code:

```
import pandas as pd

df = pd.read_csv("hospital.csv")

df["Age"] = pd.to_numeric(df["Age"], errors="coerce")

df["TreatmentCost"] = pd.to_numeric(
    df["TreatmentCost"], errors="coerce"
)

df["AdmissionDate"] = pd.to_datetime(
    df["AdmissionDate"],
    errors="coerce",
    dayfirst=True
)

df["DischargeDate"] = pd.to_datetime(
    df["DischargeDate"],
    errors="coerce",
    dayfirst=True
)

print("2. RULES FOR HANDLING MISSING AND INVALID VALUES")

print("-" * 75)

df["PatientID"] = df["PatientID"].replace(
    ["NULL", "null", ""], pd.NA
)

df.loc[
    (df["Age"] <= 0) | (df["Age"] > 120),
    "Age"
] = pd.NA

df["Age"] = df["Age"].fillna(
    df["Age"].median()
```

```

)

df["TreatmentCost"] = df["TreatmentCost"].fillna(

    df["TreatmentCost"].median()

)

invalid_date = (

    df["DischargeDate"] < df["AdmissionDate"]

)

df.loc[invalid_date, "DischargeDate"] = pd.NaT

print("\nCleaned Dataset:")

print(df.to_string(index=False))

print("\nRemaining Missing Values:")

print(df.isnull().sum())

```

Output:

```

2. RULES FOR HANDLING MISSING AND INVALID VALUES
-----

Cleaned Dataset:
PatientID      Name  Age Gender AdmissionDate DischargeDate  Diagnosis  TreatmentCost DoctorID
P1001  Ravi Kumar  45.0      M    2025-01-10    2025-01-15    Diabetes      25000.0      D101
P1002    Priya S  38.0      F    2025-01-12              NaT Hypertension      18000.0      D102
P1003    Arun V  39.0      M    2025-01-13    2025-01-20    diabetes      22000.0      D101
   NaN    Kavya R  29.0      F    2025-01-15    2025-01-17    Diabtes      15000.0      D103
P1005    Rahul P  51.0      M    2025-01-18    2025-01-25    Asthma      16500.0      D104
P1005    Rahul P  51.0      M    2025-01-18    2025-01-25    Asthma      16500.0      D104
P1006    Sneha M  34.0      F    2025-01-15    2025-01-18 Hypertension      12000.0      D102
P1007    Deepak N  39.0      M    2025-02-01    2025-02-05    Asthma      10000.0      D104
P1008    Meena K  26.0      F    2025-02-10    2025-02-15  HYPERTENSION      13500.0      D102
P1009    John D  40.0      M    2025-02-12    2025-02-18    Diabetes      20000.0      D101

Remaining Missing Values:
PatientID      1
Name           0
Age            0
Gender         0
AdmissionDate  0
DischargeDate  1
Diagnosis      0
TreatmentCost  0
DoctorID       0
dtype: int64

```

3. Create a process to standardize diagnosis names. (4 Marks)

Code:

```

import pandas as pd

df = pd.read_csv("hospital.csv")

print("3. DIAGNOSIS STANDARDIZATION")

print("-" * 60)

print("\nOriginal Diagnosis Values:")

print(df["Diagnosis"].unique())

df["Diagnosis"] = (

    df["Diagnosis"]

    .astype(str)

    .str.strip()

    .str.lower()

)

diagnosis_mapping = {

    "diabetes": "Diabetes",

    "diabtes": "Diabetes",

    "hypertension": "Hypertension",

    "asthma": "Asthma"

}

df["Diagnosis"] = df["Diagnosis"].replace(

    diagnosis_mapping

)

print("\nStandardized Diagnosis Values:")

print(df["Diagnosis"].unique())

print("\nStandardized Dataset:")

print(

    df[

        ["PatientID", "Name", "Diagnosis"]

    ].to_string(index=False)

)

```


Output:

3. DIAGNOSIS STANDARDIZATION

Original Diagnosis Values:

```
<StringArray>  
['Diabetes', 'Hypertension', 'diabetes', 'Diabtes', 'Asthma', 'HYPERTENSION']  
Length: 6, dtype: str
```

Standardized Diagnosis Values:

```
<StringArray>  
['Diabetes', 'Hypertension', 'Asthma']  
Length: 3, dtype: str
```

Standardized Dataset:

PatientID	Name	Diagnosis
P1001	Ravi Kumar	Diabetes
P1002	Priya S	Hypertension
P1003	Arun V	Diabetes
NaN	Kavya R	Diabetes
P1005	Rahul P	Asthma
P1005	Rahul P	Asthma
P1006	Sneha M	Hypertension
P1007	Deepak N	Asthma
P1008	Meena K	Hypertension
P1009	John D	Diabetes

4. Evaluate how cleaning changes average treatment cost and hospital stay duration. (4 Marks)

Code:

```
import pandas as pd  
  
df = pd.read_csv("hospital.csv")  
  
df["TreatmentCost"] = pd.to_numeric(  
    df["TreatmentCost"], errors="coerce"  
)  
  
df["AdmissionDate"] = pd.to_datetime(  
    df["AdmissionDate"],  
    errors="coerce",  
    dayfirst=True  
)
```

```

df["DischargeDate"] = pd.to_datetime(
    df["DischargeDate"],
    errors="coerce",
    dayfirst=True
)

df["StayDuration_Before"] = (
    df["DischargeDate"] - df["AdmissionDate"]
).dt.days

avg_cost_before = df["TreatmentCost"].mean()
avg_stay_before = df["StayDuration_Before"].mean()

print("4. ANALYSIS BEFORE CLEANING")

print("-" * 60)

print("Average Treatment Cost:",
      round(avg_cost_before, 2))

print("Average Hospital Stay:",
      round(avg_stay_before, 2), "days")

cleaned_df = df.copy()

cleaned_df = cleaned_df.drop_duplicates()

cleaned_df.loc[
    cleaned_df["TreatmentCost"] < 0,
    "TreatmentCost"
] = pd.NA

cleaned_df["TreatmentCost"] = cleaned_df[
    "TreatmentCost"
].fillna(
    cleaned_df["TreatmentCost"].median()
)

cleaned_df["Age"] = pd.to_numeric(
    cleaned_df["Age"], errors="coerce"
)

```

```

)

cleaned_df.loc[
    cleaned_df["Age"] > 120,
    "Age"
] = pd.NA

cleaned_df.loc[
    cleaned_df["DischargeDate"] <
    cleaned_df["AdmissionDate"],
    "DischargeDate"
] = pd.NaT

cleaned_df["StayDuration_After"] = (
    cleaned_df["DischargeDate"] -
    cleaned_df["AdmissionDate"]
).dt.days

avg_cost_after = cleaned_df["TreatmentCost"].mean()

avg_stay_after = cleaned_df[
    "StayDuration_After"
].mean()

print("\nANALYSIS AFTER CLEANING")

print("-" * 60)

print("Average Treatment Cost:",
      round(avg_cost_after, 2))

print("Average Hospital Stay:",
      round(avg_stay_after, 2), "days")

print("\nCOMPARISON")

print("-" * 60)

print("Treatment Cost Difference:",
      round(avg_cost_after - avg_cost_before, 2))

```

```
print("Hospital Stay Difference:",
      round(avg_stay_after - avg_stay_before, 2),
      "days")
```

Output:

```
=====
4. ANALYSIS BEFORE CLEANING
-----
Average Treatment Cost: 16937.5
Average Hospital Stay: 4.5 days

ANALYSIS AFTER CLEANING
-----
Average Treatment Cost: 16888.89
Average Hospital Stay: 4.88 days

COMPARISON
-----
Treatment Cost Difference: -48.61
Hospital Stay Difference: 0.38 days
```

5.Recommend data governance measures to prevent future errors. (4 Marks)

Code:

```
import pandas as pd

df = pd.read_csv("hospital.csv")

print("5. DATA GOVERNANCE REPORT")

print("-" * 80)

governance = pd.DataFrame({
    "Data Governance Measure": [
        "Mandatory Patient ID",
        "Data Validation Rules",
        "Duplicate Detection",
        "Date Validation",
        "Age Validation",
        "Standardized Diagnosis",
        "Mandatory Treatment Cost",
```

"Regular Data Quality Audits"

],

"Recommended Action": [

"Make Patient ID mandatory and unique for every patient.",

"Validate all fields before accepting data.",

"Automatically detect and flag duplicate patient records.",

"Ensure discharge date is not earlier than admission date.",

"Allow only realistic patient ages, such as 1 to 120 years.",

"Use a predefined list of standardized diagnosis names.",

"Require treatment cost or flag missing values for review.",

"Perform regular checks to identify missing, duplicate and invalid data."

]

})

```
print(governance.to_string(index=False))
```

```
print("\n" + "=" * 80)
```

```
print("FINAL GOVERNANCE RECOMMENDATIONS")
```

```
print("=" * 80)
```

```
print("""
```

1. Patient IDs should be mandatory and unique.
2. Validation rules should be applied during data entry.
3. Duplicate records should be automatically detected.
4. Admission and discharge dates should be validated.
5. Patient age should be checked against realistic limits.
6. Diagnosis names should be selected from a standard list.
7. Missing treatment costs should be flagged for review.
8. Regular data quality audits should be performed.

```
""")
```

Output:

```

===== RESTART: C:/Users/nares/Videos/dsa0503/experiments/as3.1.2.5.py =====
5. DATA GOVERNANCE REPORT
-----
Data Governance Measure                                     Recommended Action
Mandatory Patient ID                                       Make Patient ID mandatory and unique for every patient.
Data Validation Rules                                       Validate all fields before accepting data.
Duplicate Detection                                         Automatically detect and flag duplicate patient records.
Date Validation                                             Ensure discharge date is not earlier than admission date.
Age Validation                                              Allow only realistic patient ages, such as 1 to 120 years.
Standardized Diagnosis                                     Use a predefined list of standardized diagnosis names.
Mandatory Treatment Cost                                    Require treatment cost or flag missing values for review.
Regular Data Quality Audits Perform regular checks to identify missing, duplicate and invalid data.

=====
FINAL GOVERNANCE RECOMMENDATIONS
=====

1. Patient IDs should be mandatory and unique.
2. Validation rules should be applied during data entry.
3. Duplicate records should be automatically detected.
4. Admission and discharge dates should be validated.
5. Patient age should be checked against realistic limits.
6. Diagnosis names should be selected from a standard list.
7. Missing treatment costs should be flagged for review.
8. Regular data quality audits should be performed.

```

Question 3: Banking Loan Approval Dataset

1. Identify critical issues that could impact loan decisions. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("loan.csv")
```

```
print("1. CRITICAL ISSUES AFFECTING LOAN DECISIONS")
```

```
print("-" * 70)
```

```
print("\nMissing Values:")
```

```
print(df.isnull().sum())
```

```
print("\nDuplicate Customer IDs:")
```

```
print(df[df.duplicated("CustomerID", keep=False)])
```

```
print("\nMissing Credit Scores:")
```

```
print(df[df["CreditScore"].isna()])
```

```
print("\nNegative Loan Amounts:")
```

```
print(df[df["LoanAmount"] < 0])
```

```
print("\nAge Below 18:")
```

```
print(df[df["Age"] < 18])
```

```
print("\nEmployment Status Values:")
```

```
print(df["EmploymentStatus"].unique())
```

```
print("\nIncome Values:")
```

```
print(df["AnnualIncome"])
```

Output:

```
===== RESTART: C:/Use
1. CRITICAL ISSUES AFFECTING LOAN DECISIONS
-----

Missing Values:
CustomerID      0
AnnualIncome    1
CreditScore     1
LoanAmount      0
EmploymentStatus 0
Age             0
ExistingDebt    0
LoanStatus      0
dtype: int64

Duplicate Customer IDs:
  CustomerID AnnualIncome  CreditScore  ...  Age ExistingDebt  LoanStatus
7         L008      650000      690.0  ...   33         70000      Approved
8         L008      650000      690.0  ...   33         70000      Approved

[2 rows x 8 columns]

Missing Credit Scores:
  CustomerID AnnualIncome  CreditScore  ...  Age ExistingDebt  LoanStatus
2         L003      1200000         NaN  ...   28         200000      Approved

[1 rows x 8 columns]

Negative Loan Amounts:
  CustomerID AnnualIncome  CreditScore  ...  Age ExistingDebt  LoanStatus
3         L004      150000      450.0  ...   30         50000      Rejected

[1 rows x 8 columns]

Age Below 18:
  CustomerID AnnualIncome  CreditScore  ...  Age ExistingDebt  LoanStatus
4         L005      5000000      790.0  ...   16         10000      Approved

[1 rows x 8 columns]

Employment Status Values:
<StringArray>
['Full Time', 'FT', 'fulltime', 'Part Time', 'Self Employed', 'Unemployed']
Length: 6, dtype: str
```

```
Income Values:
0      750000
1      900K
2     1200000
3     150000
4     5000000
5     450000
6     75000000
7     650000
8     650000
9         NaN
Name: AnnualIncome, dtype: str
```

2. Propose cleaning techniques for missing and inconsistent data. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("loan.csv")
```

```
df["AnnualIncome"] = (
    df["AnnualIncome"]
    .astype(str)
    .str.upper()
    .str.replace("K", "000", regex=False)
    .replace("NULL", pd.NA)
)
```

```
df["AnnualIncome"] = pd.to_numeric(
    df["AnnualIncome"], errors="coerce"
)
```

```
df["CreditScore"] = pd.to_numeric(
    df["CreditScore"], errors="coerce"
)
```



```
df["LoanAmount"] = pd.to_numeric(  
    df["LoanAmount"], errors="coerce"  
)
```

```
df["Age"] = pd.to_numeric(  
    df["Age"], errors="coerce"  
)
```

```
df["EmploymentStatus"] = (  
    df["EmploymentStatus"]  
    .astype(str)  
    .str.strip()  
    .str.lower()  
)
```

```
df["EmploymentStatus"] = df["EmploymentStatus"].replace({  
    "ft": "Full Time",  
    "fulltime": "Full Time",  
    "full time": "Full Time",  
    "part time": "Part Time",  
    "self employed": "Self Employed",  
    "unemployed": "Unemployed"  
})
```

```
df.loc[df["LoanAmount"] < 0, "LoanAmount"] = pd.NA
```

```
df.loc[df["Age"] < 18, "Age"] = pd.NA
```

```
df["AnnualIncome"] = df["AnnualIncome"].fillna(
    df["AnnualIncome"].median()
)
```

```
df["CreditScore"] = df["CreditScore"].fillna(
    df["CreditScore"].median()
)
```

```
print("2. CLEANED DATA")
print("-" * 70)
print(df.to_string(index=False))
```

Output:

```
----- RESUME: C:/Users/nales/VIDEOS/usa0005/
2. CLEANED DATA
-----
CustomerID AnnualIncome CreditScore LoanAmount EmploymentStatus Age ExistingDebt LoanStatus
L001      750000.0      720.0    300000.0      Full Time 35.0      50000    Approved
L002      900000.0      680.0    400000.0      Full Time 42.0     100000    Approved
L003     1200000.0      690.0    600000.0      Full Time 28.0     200000    Approved
L004      150000.0      450.0         NaN      Part Time 30.0      50000    Rejected
L005     5000000.0      790.0   1000000.0      Self Employed NaN      10000    Approved
L006      450000.0      350.0    250000.0      Unemployed 40.0     300000    Rejected
L007     75000000.0      800.0    500000.0      Full Time 45.0     100000    Approved
L008      650000.0      690.0    350000.0      Full Time 33.0      70000    Approved
L008      650000.0      690.0    350000.0      Full Time 33.0      70000    Approved
L010      750000.0      710.0    300000.0      Full Time 37.0      80000    Approved
```

3. Detect and justify treatment of outliers. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("loan.csv")
```

```
df["AnnualIncome"] = (
    df["AnnualIncome"]
    .astype(str)
```

```
.str.upper()

.str.replace("K", "000", regex=False)

)
```

```
df["AnnualIncome"] = pd.to_numeric(
    df["AnnualIncome"], errors="coerce"
)
```

```
Q1 = df["AnnualIncome"].quantile(0.25)
Q3 = df["AnnualIncome"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR
```

```
outliers = df[
    (df["AnnualIncome"] < lower_limit) |
    (df["AnnualIncome"] > upper_limit)
]
```

```
print("3. OUTLIER DETECTION")
print("-" * 70)
```

```
print("\nQ1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)
```

```

print("\nDetected Outliers:")

print(outliers.to_string(index=False))

print("\nOutlier Treatment:")

print("Extreme income values should be verified with the original")

print("source before removal or replacement.")

```

Output:

3. OUTLIER DETECTION

```

Q1: 650000.0
Q3: 1200000.0
IQR: 550000.0
Lower Limit: -175000.0
Upper Limit: 2025000.0

```

Detected Outliers:

CustomerID	AnnualIncome	CreditScore	LoanAmount	EmploymentStatus	Age	ExistingDebt	LoanStatus
L005	5000000.0	790.0	1000000	Self Employed	16	10000	Approved
L007	75000000.0	800.0	500000	Full Time	45	100000	Approved

Outlier Treatment:

```

Extreme income values should be verified with the original
source before removal or replacement.

```

4. Analyze the effect of cleaning on loan approval rates. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("loan.csv")
```

```
df["AnnualIncome"] = (
```

```
    df["AnnualIncome"]
```

```
    .astype(str)
```

```
    .str.upper()
```

```
    .str.replace("K", "000", regex=False)
```

)

```
df["AnnualIncome"] = pd.to_numeric(  
    df["AnnualIncome"], errors="coerce"
```

)

```
df["CreditScore"] = pd.to_numeric(  
    df["CreditScore"], errors="coerce"
```

)

```
df["LoanAmount"] = pd.to_numeric(  
    df["LoanAmount"], errors="coerce"
```

)

```
df["Age"] = pd.to_numeric(  
    df["Age"], errors="coerce"
```

)

```
df["EmploymentStatus"] = (  
    df["EmploymentStatus"]
```

```
    .astype(str)
```

```
    .str.strip()
```

```
    .str.lower()
```

```
)
```

)

```
df["EmploymentStatus"] = df["EmploymentStatus"].replace({
```

```
    "ft": "Full Time",
```

```
    "fulltime": "Full Time",
```

```
    "full time": "Full Time",
```

```
"part time": "Part Time",  
"self employed": "Self Employed",  
"unemployed": "Unemployed"  
})
```

```
approval_before = (  
    df["LoanStatus"].str.lower() == "approved"  
).mean() * 100
```

```
cleaned_df = df.drop_duplicates(  
    subset="CustomerID"  
).copy()
```

```
cleaned_df.loc[  
    cleaned_df["LoanAmount"] < 0,  
    "LoanAmount"  
] = pd.NA
```

```
cleaned_df.loc[  
    cleaned_df["Age"] < 18,  
    "Age"  
] = pd.NA
```

```
cleaned_df["AnnualIncome"] = cleaned_df[  
    "AnnualIncome"  
].fillna(  
    cleaned_df["AnnualIncome"].median()  
)
```

```

cleaned_df["CreditScore"] = cleaned_df[
    "CreditScore"
].fillna(
    cleaned_df["CreditScore"].median()
)

approval_after = (
    cleaned_df["LoanStatus"].str.lower() == "approved"
).mean() * 100

print("4. LOAN APPROVAL RATE ANALYSIS")

print("-" * 60)

print("\nApproval Rate Before Cleaning:")
print(round(approval_before, 2), "%")

print("\nApproval Rate After Cleaning:")
print(round(approval_after, 2), "%")

print("\nChange in Approval Rate:")
print(round(approval_after - approval_before, 2), "%")

```

OUTPUT:

4. LOAN APPROVAL RATE ANALYSIS

Approval Rate Before Cleaning:
80.0 %

Approval Rate After Cleaning:
77.78 %

Change in Approval Rate:
-2.22 %

5. Discuss ethical implications of incorrect data cleaning decisions. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("loan.csv")
```

```
ethical_issues = pd.DataFrame({
```

```
    "Ethical Issue": [
```

```
        "Incorrect Removal of Records",
```

```
        "Incorrect Outlier Treatment",
```

```
        "Unfair Imputation",
```

```
        "Biased Approval Decisions",
```

```
        "Loss of Customer Trust",
```

```
        "Lack of Transparency"
```

```
    ],
```

```
    "Ethical Implication": [
```

```
        "Removing valid customer records may unfairly exclude applicants from analysis.",
```

```
        "Removing genuine high-income values can incorrectly change loan eligibility.",
```

```
        "Replacing missing values with unsuitable values can affect individual applicants.",
```

```
        "Incorrect cleaning can cause some customers to be unfairly approved or rejected.",
```

```
        "Incorrect decisions based on poor data can reduce customer confidence in the bank.",
```

```
        "Customers may be affected if cleaning decisions are made without clear documentation."
```

```
    ]
```

```
})
```

```
print("5. ETHICAL IMPLICATIONS OF DATA CLEANING")
```



```
print("-" * 85)
```

```
print(ethical_issues.to_string(index=False))
```

```
print("\nFINAL ETHICAL REPORT")
```

```
print("-" * 85)
```

```
print("""
```

Incorrect data cleaning decisions can directly affect loan approval and rejection decisions. Removing valid records, changing genuine outliers, or incorrectly filling missing values can create unfair results. Therefore, cleaning rules should be consistent, documented, transparent, and verified before being used for loan decisions.

```
""")
```

Output:

```
5. ETHICAL IMPLICATIONS OF DATA CLEANING
-----
Ethical Issue                                Ethical Implication
Incorrect Removal of Records                Removing valid customer records may unfairly exclude applicants from analysis.
Incorrect Outlier Treatment                 Removing genuine high-income values can incorrectly change loan eligibility.
Unfair Imputation                         Replacing missing values with unsuitable values can affect individual applicants.
Biased Approval Decisions                  Incorrect cleaning can cause some customers to be unfairly approved or rejected.
Loss of Customer Trust                     Incorrect decisions based on poor data can reduce customer confidence in the bank.
Lack of Transparency                      Customers may be affected if cleaning decisions are made without clear documentation.

FINAL ETHICAL REPORT
-----

Incorrect data cleaning decisions can directly affect loan approval
and rejection decisions. Removing valid records, changing genuine
outliers, or incorrectly filling missing values can create unfair
results. Therefore, cleaning rules should be consistent, documented,
transparent, and verified before being used for loan decisions.
```

Question 4: Smart City Traffic Monitoring Dataset

1. Assess the quality of the traffic dataset. (4 Marks)

Code:

```
import pandas as pd

df = pd.read_csv("traffic.csv")

df["VehicleCount"] = pd.to_numeric(
    df["VehicleCount"], errors="coerce"
)

df["AverageSpeed"] = pd.to_numeric(
    df["AverageSpeed"], errors="coerce"
)

print("1. TRAFFIC DATASET QUALITY ASSESSMENT")
print("-" * 70)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Sensor Readings:")
print(df[df.duplicated(keep=False)])

print("\nNegative Vehicle Counts:")
print(df[df["VehicleCount"] < 0])

print("\nInvalid Average Speeds:")
print(df[df["AverageSpeed"] > 300])

print("\nMissing Weather Information:")
print(df[df["WeatherCondition"].isna()])
```

```
print("\nRoad Name Values:")

print(df["RoadName"].unique())
```

```
print("\nSensor ID Values:")

print(df["SensorID"].unique())
```

output:

```
-----
1. TRAFFIC DATASET QUALITY ASSESSMENT
-----

Missing Values:
SensorID          0
Timestamp         1
VehicleCount      0
AverageSpeed      0
TrafficSignalStatus 0
RoadName          0
WeatherCondition  2
dtype: int64

Duplicate Sensor Readings:
  SensorID  Timestamp  ... RoadName  WeatherCondition
5    S106  01-01-2025 08:25  ...    OMR              NaN
6    S106  01-01-2025 08:25  ...    OMR              NaN

[2 rows x 7 columns]

Negative Vehicle Counts:
  SensorID  Timestamp  ... RoadName  WeatherCondition
2    S103  01-01-2025 08:10  ... ANNA SALAI          Cloudy

[1 rows x 7 columns]

Invalid Average Speeds:
  SensorID  Timestamp  ... RoadName  WeatherCondition
3    S104  01-01-2025 08:15  ...  GST Road          Rainy

[1 rows x 7 columns]

Missing Weather Information:
  SensorID  Timestamp  ... RoadName  WeatherCondition
5    S106  01-01-2025 08:25  ...    OMR              NaN
6    S106  01-01-2025 08:25  ...    OMR              NaN

[2 rows x 7 columns]

Road Name Values:
<StringArray>
[      'Anna Salai',      'anna salai',      'ANNA SALAI',
      'GST Road',      'GST ROAD',      'OMR',
      'Old Mahabalipuram Road']
Length: 7, dtype: str
```

```
Sensor ID Values:
<StringArray>
['S101', 'S102', 'S103', 'S104', 'S105', 'S106', 'S107', 'S10A', 'S109']
Length: 9, dtype: str
```

2.Recommend cleaning techniques for time-series data. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("traffic.csv")
```

```
df["Timestamp"] = pd.to_datetime(
    df["Timestamp"],
    errors="coerce",
    dayfirst=True
)
```

```
df["VehicleCount"] = pd.to_numeric(
    df["VehicleCount"],
    errors="coerce"
)
```

```
df["AverageSpeed"] = pd.to_numeric(
    df["AverageSpeed"],
    errors="coerce"
)
```

```
df = df.drop_duplicates()
```

```
df.loc[
```

```
df["VehicleCount"] < 0,  
    "VehicleCount"  
] = pd.NA
```

```
df.loc[  
    df["AverageSpeed"] > 300,  
    "AverageSpeed"  
] = pd.NA
```

```
df["RoadName"] = (  
    df["RoadName"]  
    .str.strip()  
    .str.title()  
)
```

```
df["Timestamp"] = df["Timestamp"].interpolate(  
    method="linear"  
)
```

```
df["WeatherCondition"] = df[  
    "WeatherCondition"  
].fillna("Unknown")
```

```
print("2. TIME-SERIES DATA CLEANING")  
print("-" * 70)  
print(df.to_string(index=False))
```

Output:

===== RESTART: C:/Users/nares/Videos/dsa0503/experiments/as3.1.4.2.py =====

2. TIME-SERIES DATA CLEANING

```
-----
SensorID      Timestamp  VehicleCount  AverageSpeed  TrafficSignalStatus  RoadName  WeatherCondition
S101 2025-01-01 08:00:00      120.0         45.0           Green        Anna Salai      Sunny
S102 2025-01-01 08:05:00       95.0         52.0           Red          Anna Salai      Sunny
S103 2025-01-01 08:10:00       NaN          48.0           Green        Anna Salai      Cloudy
S104 2025-01-01 08:15:00      140.0         NaN            Yellow       Gst Road        Rainy
S105 2025-01-01 08:20:00      110.0         42.0           Green       Gst Road        Sunny
S106 2025-01-01 08:25:00      130.0         46.0           Red          Omr            Unknown
S107 2025-01-01 08:30:00       85.0         55.0           Green Old Mahabalipuram Road Cloudy
S10A 2025-01-01 08:35:00      100.0         50.0           Red          Anna Salai      Sunny
S109 2025-01-01 08:40:00      125.0         47.0           Green       Gst Road        Rainy
```

3.Design rules to identify faulty sensor readings. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("traffic.csv")
```

```
df["Timestamp"] = pd.to_datetime(
```

```
    df["Timestamp"],
```

```
    errors="coerce",
```

```
    dayfirst=True
```

```
)
```

```
df["VehicleCount"] = pd.to_numeric(
```

```
    df["VehicleCount"],
```

```
    errors="coerce"
```

```
)
```

```
df["AverageSpeed"] = pd.to_numeric(
```

```
    df["AverageSpeed"],
```

```
    errors="coerce"
```

```
)
```

```
faulty = pd.DataFrame()
```

```
faulty["Missing Timestamp"] = df["Timestamp"].isna()
```

```
faulty["Negative Vehicle Count"] = df["VehicleCount"] < 0
```

```
faulty["Speed Above 300"] = df["AverageSpeed"] > 300
```

```
faulty["Duplicate Reading"] = df.duplicated(
```

```
    keep=False
```

```
)
```

```
faulty["Faulty Reading"] = faulty.any(axis=1)
```

```
result = df.copy()
```

```
result["Faulty Reading"] = faulty["Faulty Reading"]
```

```
result["Reason"] = ""
```

```
result.loc[
```

```
    faulty["Missing Timestamp"],
```

```
    "Reason"
```

```
] += "Missing Timestamp; "
```

```
result.loc[
```

```
    faulty["Negative Vehicle Count"],
```

```
    "Reason"
```

```
] += "Negative Vehicle Count; "
```

```
result.loc[
```

```
    faulty["Speed Above 300"],
```

```

        "Reason"

    ] += "Speed Above 300 km/h; "

result.loc[

    faulty["Duplicate Reading"],

    "Reason"

] += "Duplicate Reading; "

print("3. FAULTY SENSOR READING DETECTION")

print("-" * 75)

print(

    result[result["Faulty Reading"]].to_string(

        index=False

    )

)

```

Output:

```

===== RESTART: C:/Users/nares/Videos/dsa0503/experiments/as3.1.4.3.py =====
3. FAULTY SENSOR READING DETECTION
-----
SensorID      Timestamp  VehicleCount  AverageSpeed  TrafficSignalStatus  RoadName  WeatherCondition  Faulty Reading  Reason
S103 2025-01-01 08:10:00      -50          48          Green  ANNA SALAI      Cloudy      True  Negative Vehicle Count;
S104 2025-01-01 08:15:00      140         350          Yellow  GST Road      Rainy      True  Speed Above 300 km/h;
S105                NaT      110          42          Green  GST ROAD      Sunny      True  Missing Timestamp;
S106 2025-01-01 08:25:00      130          46          Red      OMR      NaN      True  Duplicate Reading;
S106 2025-01-01 08:25:00      130          46          Red      OMR      NaN      True  Duplicate Reading;

```

4.Evaluate how cleaning affects congestion analysis. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("traffic.csv")
```



```
df["VehicleCount"] = pd.to_numeric(  
    df["VehicleCount"],  
    errors="coerce"  
)
```

```
df["AverageSpeed"] = pd.to_numeric(  
    df["AverageSpeed"],  
    errors="coerce"  
)
```

```
df["Timestamp"] = pd.to_datetime(  
    df["Timestamp"],  
    errors="coerce",  
    dayfirst=True  
)
```

```
df["CongestionBefore"] = (  
    (df["VehicleCount"] > 130) |  
    (df["AverageSpeed"] < 45)  
)
```

```
before = df["CongestionBefore"].sum()
```

```
cleaned_df = df.drop_duplicates().copy()
```

```
cleaned_df.loc[  
    cleaned_df["VehicleCount"] < 0,
```

```

    "VehicleCount"
] = pd.NA

cleaned_df.loc[
    cleaned_df["AverageSpeed"] > 300,
    "AverageSpeed"
] = pd.NA

cleaned_df["CongestionAfter"] = (
    (cleaned_df["VehicleCount"] > 130) |
    (cleaned_df["AverageSpeed"] < 45)
)

after = cleaned_df["CongestionAfter"].sum()

total_before = len(df)
total_after = len(cleaned_df)

rate_before = (before / total_before) * 100
rate_after = (after / total_after) * 100

print("4. CONGESTION ANALYSIS")
print("-" * 70)

print("\nBefore Cleaning:")
print("Congested Readings:", before)
print("Total Readings:", total_before)

```

```
print("Congestion Rate:", round(rate_before, 2), "%")
```

```
print("\nAfter Cleaning:")
```

```
print("Congested Readings:", after)
```

```
print("Total Readings:", total_after)
```

```
print("Congestion Rate:", round(rate_after, 2), "%")
```

```
print("\nChange in Congestion Rate:")
```

```
print(round(rate_after - rate_before, 2), "%")
```

Output:

4. CONGESTION ANALYSIS

Before Cleaning:

Congested Readings: 2

Total Readings: 10

Congestion Rate: 20.0 %

After Cleaning:

Congested Readings: 2

Total Readings: 9

Congestion Rate: 22.22 %

Change in Congestion Rate:

2.22 %

5.Suggest automated monitoring techniques for future data collection. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("traffic.csv")
```

```
df["VehicleCount"] = pd.to_numeric(
```

```
    df["VehicleCount"], errors="coerce"
```

```
)
```

```
df["AverageSpeed"] = pd.to_numeric(  
    df["AverageSpeed"], errors="coerce"  
)
```

```
df["Timestamp"] = pd.to_datetime(  
    df["Timestamp"], errors="coerce", dayfirst=True  
)
```

```
df["MissingTimestamp"] = df["Timestamp"].isna()
```

```
df["NegativeVehicleCount"] = (  
    df["VehicleCount"] < 0  
)
```

```
df["InvalidSpeed"] = (  
    df["AverageSpeed"] > 300  
)
```

```
df["DuplicateReading"] = df.duplicated(  
    keep=False  
)
```

```
df["MissingWeather"] = (  
    df["WeatherCondition"].isna()  
)
```

```
df["MonitoringStatus"] = "Valid"
```

```
df.loc[
```

```
    df["MissingTimestamp"] |
```

```
    df["NegativeVehicleCount"] |
```

```
    df["InvalidSpeed"] |
```

```
    df["DuplicateReading"] |
```

```
    df["MissingWeather"],
```

```
    "MonitoringStatus"
```

```
] = "Alert"
```

```
print("5. AUTOMATED DATA MONITORING")
```

```
print("-" * 70)
```

```
print(df[
```

```
    [
```

```
        "SensorID",
```

```
        "Timestamp",
```

```
        "VehicleCount",
```

```
        "AverageSpeed",
```

```
        "MonitoringStatus"
```

```
    ]
```

```
].to_string(index=False))
```

```
print("\nMonitoring Summary:")
```

```
print(df["MonitoringStatus"].value_counts())
```

Output:

```
===== RESTART: C:/Users/.../Python310/Scripts/python.exe
5. AUTOMATED DATA MONITORING
-----
SensorID      Timestamp      VehicleCount  AverageSpeed  MonitoringStatus
S101 2025-01-01 08:00:00      120           45           Valid
S102 2025-01-01 08:05:00       95           52           Valid
S103 2025-01-01 08:10:00      -50           48           Alert
S104 2025-01-01 08:15:00      140          350           Alert
S105                    NaT      110           42           Alert
S106 2025-01-01 08:25:00      130           46           Alert
S106 2025-01-01 08:25:00      130           46           Alert
S107 2025-01-01 08:30:00       85           55           Valid
S10A 2025-01-01 08:35:00      100           50           Valid
S109 2025-01-01 08:40:00      125           47           Valid

Monitoring Summary:
MonitoringStatus
Valid      5
Alert      5
Name: count, dtype: int64
|
```

Question 5: Global Sales Performance Dataset

1. Develop a comprehensive data-cleaning framework for multinational datasets. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("sales.csv")
```

```
df["SalesAmount"] = pd.to_numeric(
    df["SalesAmount"],
    errors="coerce"
)
```

```
df["CustomerRating"] = pd.to_numeric(
    df["CustomerRating"],
    errors="coerce"
)
```

```
df["SalesDate"] = pd.to_datetime(  
    df["SalesDate"],  
    errors="coerce",  
    dayfirst=True  
)
```

```
df["Currency"] = (  
    df["Currency"]  
    .astype(str)  
    .str.strip()  
    .str.upper()  
)
```

```
df["Region"] = (  
    df["Region"]  
    .astype(str)  
    .str.strip()  
    .str.title()  
)
```

```
df["Country"] = (  
    df["Country"]  
    .astype(str)  
    .str.strip()  
    .str.title()  
)
```

```

df["SalesPerson"] = (
    df["SalesPerson"]
    .astype(str)
    .str.strip()
)

df["SalesPerson"] = df["SalesPerson"].replace({
    "Rajkumar": "Raj Kumar"
})

df["Product"] = (
    df["Product"]
    .astype(str)
    .str.strip()
)

df = df.drop_duplicates()

df.loc[
    (df["CustomerRating"] < 1) |
    (df["CustomerRating"] > 5),
    "CustomerRating"
] = pd.NA

print("1. DATA CLEANING FRAMEWORK")

print("-" * 75)

```



```
print("\nCleaned Dataset:")

print(df.to_string(index=False))

print("\nMissing Values:")

print(df.isnull().sum())

print("\nDuplicate Records:")

print(df.duplicated().sum())
```

Output:

```
===== RESTART: C:/Users/nares/videos/dsa0503/experi
1. DATA CLEANING FRAMEWORK
-----

Cleaned Dataset:
   Region Country SalesPerson      Product  SalesAmount  Currency  SalesDate  CustomerRating
0     Asia   India    Raj Kumar      Laptop      50000.0      INR 2025-01-10              4.0
1     Asia   India    Raj Kumar      Laptop      55000.0      INR 2025-01-10              5.0
2  Europe Germany    Anna M      Laptop      1200.0      EUR 2025-01-12              4.0
3 North America  Usa      John D      Laptop      1500.0      USD              NaT              NaN
4     Asia   India    Priya S      Mobile              NaN      INR 2025-01-18              3.0
5  Europe France  Pierre L Ordinateur Portable      1300.0      EUR 2025-01-20              4.0
6     Asia   India    Rahul P      Mobile      9999999.0      INR 2025-01-25              5.0
7      Uk      Uk    David T      Laptop      1100.0      GBP 2025-01-28              2.0
8     Asia   India    Priya S      Smartphone      35000.0      INR 2025-01-30              NaN

Missing Values:
Region          0
Country         0
SalesPerson     0
Product         0
SalesAmount     1
Currency        0
SalesDate       1
CustomerRating  2
dtype: int64

Duplicate Records:
0
```

2.Design methods to standardize currencies, products, and dates. (4 Marks)

Code:

```
import pandas as pd

df = pd.read_csv("sales.csv")
```

```
df["Currency"] = (  
    df["Currency"]  
    .astype(str)  
    .str.strip()  
    .str.upper()  
)  
  
currency_mapping = {  
    "INR": "INR",  
    "EUR": "EUR",  
    "USD": "USD",  
    "GBP": "GBP"  
}  
  
df["Currency"] = df["Currency"].replace(currency_mapping)  
  
product_mapping = {  
    "Ordinateur Portable": "Laptop",  
    "Laptop": "Laptop",  
    "Mobile": "Mobile",  
    "Smartphone": "Smartphone"  
}  
  
df["Product"] = (  
    df["Product"]  
    .astype(str)  
    .str.strip()  
    .replace(product_mapping)  
)  
  
df["SalesDate"] = pd.to_datetime(  
    df["SalesDate"],
```

```
errors="coerce",  
dayfirst=False  
)  
  
print("2. STANDARDIZATION OF CURRENCIES, PRODUCTS AND DATES")  
  
print("-" * 75)  
  
print("\nStandardized Dataset:")  
  
print(df.to_string(index=False))  
  
print("\nCurrency Values:")  
  
print(df["Currency"].unique())  
  
print("\nProduct Values:")  
  
print(df["Product"].unique())  
  
print("\nStandardized Dates:")  
  
print(df["SalesDate"])
```

Output:

2. STANDARDIZATION OF CURRENCIES, PRODUCTS AND DATES

Standardized Dataset:

	Region	Country	SalesPerson	Product	SalesAmount	Currency	SalesDate	CustomerRating
	Asia	India	Raj Kumar	Laptop	50000.0	INR	2025-10-01	4
	Asia	India	Rajkumar	Laptop	55000.0	INR	2025-10-01	5
	Europe	Germany	Anna M	Laptop	1200.0	EUR	2025-12-01	4
	Europe	Germany	Anna M	Laptop	1200.0	EUR	2025-12-01	4
North America		USA	John D	Laptop	1500.0	USD	2025-01-15	6
	Asia	India	Priya S	Mobile	NaN	INR	NaT	3
	Europe	France	Pierre L	Laptop	1300.0	EUR	NaT	4
	Asia	India	Rahul P	Mobile	9999999.0	INR	NaT	5
	UK	UK	David T	Laptop	1100.0	GBP	NaT	2
	Asia	India	Priya S	Smartphone	35000.0	INR	NaT	0

Currency Values:

```
<StringArray>  
['INR', 'EUR', 'USD', 'GBP']  
Length: 4, dtype: str
```

Product Values:

```
<StringArray>  
['Laptop', 'Mobile', 'Smartphone']  
Length: 3, dtype: str
```

Standardized Dates:

```
0    2025-10-01  
1    2025-10-01  
2    2025-12-01  
3    2025-12-01  
4    2025-01-15  
5           NaT  
6           NaT  
7           NaT  
8           NaT  
9           NaT  
Name: SalesDate, dtype: datetime64[us]
```

3. Handle missing values and outliers using appropriate analytical techniques. (4 Marks)

Code:

```
import pandas as pd
```

```
df = pd.read_csv("sales.csv")
```

```
df["SalesAmount"] = pd.to_numeric(
```

```
    df["SalesAmount"],
```

```
    errors="coerce"
```

```
)
```

```
median_sales = df["SalesAmount"].median()
```

```
df["SalesAmount"] = df["SalesAmount"].fillna(  
    median_sales  
)
```

```
Q1 = df["SalesAmount"].quantile(0.25)
```

```
Q3 = df["SalesAmount"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_limit = Q1 - 1.5 * IQR
```

```
upper_limit = Q3 + 1.5 * IQR
```

```
outliers = df[  
    (df["SalesAmount"] < lower_limit) |  
    (df["SalesAmount"] > upper_limit)  
]
```

```
df["Outlier"] = (  
    (df["SalesAmount"] < lower_limit) |  
    (df["SalesAmount"] > upper_limit)  
)
```

```
df.loc[  
    df["Outlier"],
```

```

    "SalesAmount"

] = median_sales

print("3. MISSING VALUES AND OUTLIER TREATMENT")

print("-" * 75)

print("\nMedian Sales Amount:")

print(median_sales)

print("\nDetected Outliers:")

print(outliers.to_string(index=False))

print("\nCleaned Dataset:")

print(df.to_string(index=False))

```

Output:

```

-----
3. MISSING VALUES AND OUTLIER TREATMENT
-----

Median Sales Amount:
1500.0

Detected Outliers:
Region Country SalesPerson Product SalesAmount Currency SalesDate CustomerRating
Asia India Rahul P Mobile 9999999.0 INR 25-01-2025 5

Cleaned Dataset:
Region Country SalesPerson Product SalesAmount Currency SalesDate CustomerRating Outlier
Asia India Raj Kumar Laptop 50000.0 INR 10-01-2025 4 False
Asia India Rajkumar Laptop 55000.0 INR 10-01-2025 5 False
Europe Germany Anna M Laptop 1200.0 EUR 12-01-2025 4 False
Europe Germany Anna M Laptop 1200.0 EUR 12-01-2025 4 False
North America USA John D Laptop 1500.0 USD 01-15-2025 6 False
Asia India Priya S Mobile 1500.0 INR 18-01-2025 3 False
Europe France Pierre L Ordinateur Portable 1300.0 EUR 20-01-2025 4 False
Asia India Rahul P Mobile 1500.0 INR 25-01-2025 5 True
UK UK David T Laptop 1100.0 GBP 28-01-2025 2 False
Asia India Priya S Smartphone 35000.0 INR 30-01-2025 0 False

```

4. Compare regional sales rankings before and after cleaning. (4 Marks)

Code:

```
import pandas as pd
```

```

df = pd.read_csv("sales.csv")

df["SalesAmount"] = pd.to_numeric(
    df["SalesAmount"],
    errors="coerce"
)

df["CustomerRating"] = pd.to_numeric(
    df["CustomerRating"],
    errors="coerce"
)

sales_before = (
    df.groupby("Region")["SalesAmount"]
    .sum()
    .sort_values(ascending=False)
)

cleaned_df = df.drop_duplicates().copy()
median_sales = cleaned_df["SalesAmount"].median()
cleaned_df["SalesAmount"] = cleaned_df[
    "SalesAmount"
].fillna(median_sales)

Q1 = cleaned_df["SalesAmount"].quantile(0.25)
Q3 = cleaned_df["SalesAmount"].quantile(0.75)
IQR = Q3 - Q1
upper_limit = Q3 + 1.5 * IQR
cleaned_df.loc[
    cleaned_df["SalesAmount"] > upper_limit,
    "SalesAmount"
] = median_sales

```

```

cleaned_df.loc[
    (cleaned_df["CustomerRating"] < 1) |
    (cleaned_df["CustomerRating"] > 5),
    "CustomerRating"
] = pd.NA

sales_after = (
    cleaned_df.groupby("Region")["SalesAmount"]
    .sum()
    .sort_values(ascending=False)
)

before_rank = sales_before.rank(
    ascending=False,
    method="min"
).astype(int)

after_rank = sales_after.rank(
    ascending=False,
    method="min"
).astype(int)

comparison = pd.DataFrame({
    "Sales Before Cleaning": sales_before,
    "Rank Before": before_rank,
    "Sales After Cleaning": sales_after,
    "Rank After": after_rank
})

comparison = comparison.sort_values(
    "Rank After"
)

```



```

print("4. REGIONAL SALES RANKING")

print("-" * 75)

print("\nRegional Sales Before Cleaning:")

print(sales_before)

print("\nRegional Sales After Cleaning:")

print(sales_after)

print("\nRanking Comparison:")

print(comparison)

```

Output:

```

4. REGIONAL SALES RANKING
-----

Regional Sales Before Cleaning:
Region
Asia          10139999.0
Europe         3700.0
North America  1500.0
UK             1100.0
Name: SalesAmount, dtype: float64

Regional Sales After Cleaning:
Region
Asia          176500.0
Europe         2500.0
North America  1500.0
UK             1100.0
Name: SalesAmount, dtype: float64

Ranking Comparison:
              Sales Before Cleaning  ...  Rank After
Region                               ...
Asia          10139999.0  ...           1
Europe         3700.0  ...           2
North America  1500.0  ...           3
UK             1100.0  ...           4

[4 rows x 4 columns]

```

5.Prepare an executive report explaining how data quality influences strategic business decisions. (4 Marks)

Code:

```

import pandas as pd

df = pd.read_csv("sales.csv")

report = pd.DataFrame({
    "Data Quality Issue": [
        "Multiple Currencies",
        "Missing Sales Amount",
        "Duplicate Entries",
        "Invalid Customer Ratings",
        "Different Date Formats",
        "Multilingual Product Names",
        "Salesperson Name Variations",
        "Extreme Sales Outlier"
    ],
    "Impact on Business Decision": [
        "Incorrect regional sales comparison and financial reporting.",
        "Incomplete revenue analysis and inaccurate sales forecasts.",
        "Sales performance may be overstated.",
        "Customer satisfaction analysis may become unreliable.",
        "Incorrect time-based sales trends and forecasting.",
        "Product performance may be split across different names.",
        "Salesperson performance may be counted separately.",
        "Regional and product performance may be significantly distorted."
    ]
})

print("5. EXECUTIVE DATA QUALITY REPORT")

print("-" * 85)

print(report.to_string(index=False))

```

```
print("\nEXECUTIVE SUMMARY")
```

```
print("-" * 85)
```

```
print("""
```

Data quality has a direct influence on strategic business decisions.

Inaccurate sales amounts, duplicate records, inconsistent currencies,

invalid ratings, different date formats and product-name variations

can produce misleading business reports.

Poor-quality data can result in incorrect regional performance

comparisons, unreliable sales forecasts, incorrect product decisions

and inaccurate evaluation of salespersons.

Therefore, multinational sales data should be cleaned, standardized

and regularly monitored before it is used for strategic planning,

performance evaluation and forecasting.

```
""")
```

Code:

```
===== RESTART: C:/Users/nares/Videos/ds
5. EXECUTIVE DATA QUALITY REPORT
-----
      Data Quality Issue                                Impact on Business Decision
Multiple Currencies      Incorrect regional sales comparison and financial reporting.
Missing Sales Amount     Incomplete revenue analysis and inaccurate sales forecasts.
Duplicate Entries        Sales performance may be overstated.
Invalid Customer Ratings Customer satisfaction analysis may become unreliable.
Different Date Formats   Incorrect time-based sales trends and forecasting.
Multilingual Product Names Product performance may be split across different names.
Salesperson Name Variations Salesperson performance may be counted separately.
Extreme Sales Outlier    Regional and product performance may be significantly distorted.

EXECUTIVE SUMMARY
-----

Data quality has a direct influence on strategic business decisions.
Inaccurate sales amounts, duplicate records, inconsistent currencies,
invalid ratings, different date formats and product-name variations
can produce misleading business reports.

Poor-quality data can result in incorrect regional performance
comparisons, unreliable sales forecasts, incorrect product decisions
and inaccurate evaluation of salespersons.

Therefore, multinational sales data should be cleaned, standardized
and regularly monitored before it is used for strategic planning,
performance evaluation and forecasting.
```